SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA0301
COURSE NAME: Reinforcement learning

COURSE OUTCOME COVERED

CO4: *Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization.(BL3,BL4,BL5)*

Assessment Tool Weightage: 40%

Dr G. A. SENTHIL

QUESTIONS: 25 **Total Marks: 50**

Annexure A

Design and Develop Model

DURATION: 4 HRS (2 Sessions)

Sampling-Based Planning

1. Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

(5 Marks)

Flowchart Illustrating the Sampling-Based Planning Process in Model-Based Reinforcement Learning

1. Introduction

Model-Based Reinforcement Learning (MBRL) is a reinforcement learning approach in which an agent uses a model of the environment to predict the results of its actions. Instead of directly trying actions in the real environment repeatedly, the agent can use the learned or known model to simulate possible future situations and select a suitable action.

Sampling-based planning is a planning technique where the agent does not examine every possible future. Instead, it samples a limited number of possible actions or action sequences, predicts their outcomes using the environment model, evaluates them, and selects the most promising one.

The basic idea is:

Sample → Predict → Generate Trajectories → Evaluate → Select → Execute

This technique is particularly useful when the state space or action space is very large or continuous.

2. Sampling-Based Planning Process

The sampling-based planning process consists of the following major steps.

Step 1: Observe the Current State

The agent first observes the current state of the environment.

The state may contain information such as:

- Position of the agent
- Speed
- Nearby objects
- Current goal
- Available resources
- Previous actions

For example, in an autonomous vehicle, the current state can include the vehicle's position, speed, direction, traffic conditions, and nearby vehicles.

Step 2: Sample Possible Actions

The planner generates a set of possible actions or action sequences.

For example, a robot may have the following possible actions:

- Move forward
- Move backward
- Turn left
- Turn right

Instead of examining every possible combination, the planner selects a limited number of samples.

Mathematically, actions can be represented as:

$$a_t \sim \pi(a \mid s_t)$$

where:

- a_t= sampled action
- s_t= current state
- π= policy used for sampling

Step 3: Use the Environment Model

The sampled actions are given to the **environment model**.

The environment model predicts what will happen when an action is taken.

It can be represented as:

$$s_{t+1} = f(s_t, a_t)$$

where:

- s_t= current state
- a_t= selected action
- s_{t+1}= predicted next state
- f= environment model

For example, if a robot moves forward, the model predicts its future position.

Step 4: Generate Sampled Trajectories

A sequence of predicted states and actions forms a **trajectory**.

For example:

$$s_0 \to a_0 \to s_1 \to a_1 \to s_2$$

Several such trajectories are generated:

$$\tau_1, \tau_2, \tau_3, \dots, \tau_N$$

Each trajectory represents one possible future of the agent.

Step 5: Evaluate the Trajectories

Each trajectory is evaluated using a reward or cost function.

The total return can be calculated as:

$$G = \sum_{t=0}^{T-1} \gamma^t r_t$$

where:

- G= total return
- r_t= reward received at time t
- γ= discount factor
- T= planning horizon

A trajectory that reaches the goal safely and efficiently receives a higher reward.

Step 6: Select the Best Trajectory

After evaluating all sampled trajectories, the planner compares their scores.

The trajectory with the highest expected return is selected.

For example:

Trajectory	Expected Reward
Trajectory 1	50
Trajectory 2	85
Trajectory 3	65

Here, **Trajectory 2** is selected because it has the highest expected reward.

Step 7: Execute the Best Action

The planner generally executes the **first action** of the selected trajectory in the real environment.

After execution, the environment changes and the agent receives a new state.

The planner then repeats the entire process.

This is known as **receding-horizon planning** or **model predictive control-style planning**.

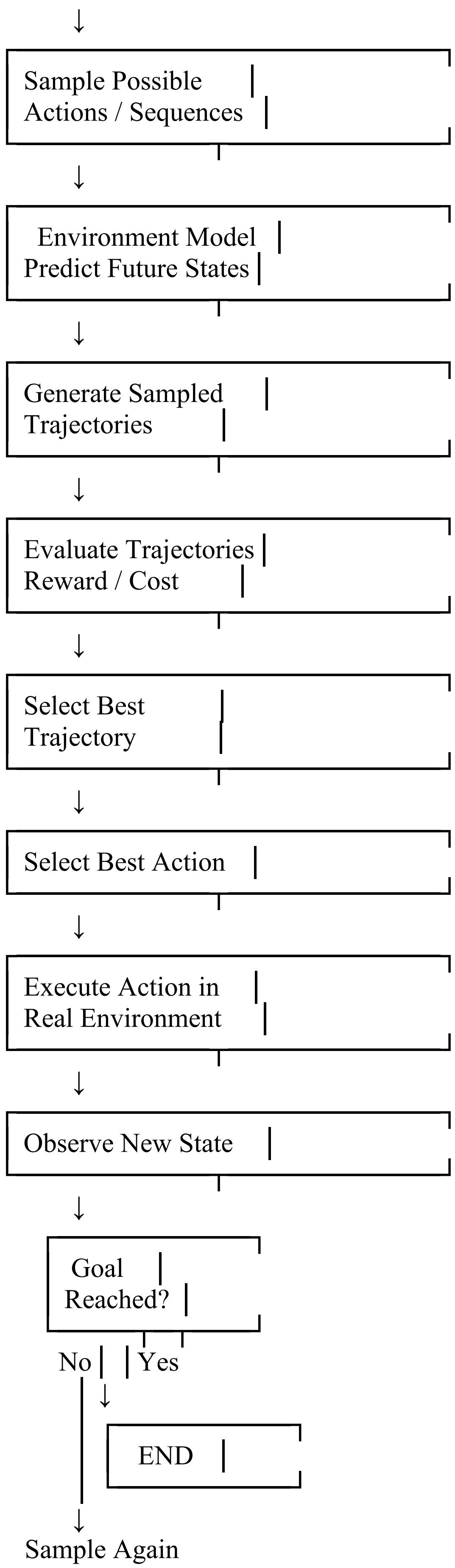
3. Flowchart

The sampling-based planning process can be represented using the following flowchart:

START

↓

Observe Current State
S_t


Sample Possible Actions / Sequences
Environment Model Predict Future States
Generate Sampled Trajectories
Evaluate Trajectories Reward / Cost
Select Best Trajectory
Select Best Action
Execute Action in Real Environment
Observe New State
Goal Reached?
No
Yes
END
Sample Again

4. Real-Life Example – Autonomous Vehicle

Consider an **autonomous car** that needs to reach a destination.

The car first observes its current state, including its speed, position, road conditions, traffic signals, and nearby vehicles.

The planner samples different possible actions:

- Continue straight
- Slow down
- Change to the left lane
- Change to the right lane

The environment model predicts the future state for each action. The planner then generates different possible driving trajectories.

Each trajectory is evaluated according to:

- Safety
- Distance
- Travel time
- Traffic conditions
- Fuel or energy consumption

Suppose the planner produces three possible trajectories:

Trajectory 1 → High traffic → Reward = 40
Trajectory 2 → Safe and efficient → Reward = 90
Trajectory 3 → Short but risky → Reward = 30

The planner selects **Trajectory 2** because it provides the highest expected reward.

The car executes the first action from this trajectory. After moving, it observes its new state and performs the planning process again.

5. Advantages of Sampling-Based Planning

1. Reduced Search Space

It does not need to examine every possible action sequence.

2. Suitable for Large Problems

It can be applied to environments with very large state and action spaces.

3. Supports Continuous Actions

Sampling works well when actions are continuous, such as steering angle or robot movement.

4. Faster Planning

By evaluating only selected samples, planning can be faster than exhaustive search.

5. Future Prediction

The environment model allows the agent to predict possible future outcomes before taking an action.

6. Adaptability

The agent can replan whenever a new state is observed.

7. Limitations

Sampling-based planning also has some limitations:

- The best possible trajectory may not be sampled.
- A large number of samples may be required for good performance.
- Simulation can be computationally expensive.
- Poor environment models can produce inaccurate predictions.
- Random sampling can sometimes produce inconsistent results.

8. Conclusion

Sampling-based planning in Model-Based Reinforcement Learning provides an efficient way to make decisions by **sampling possible actions, predicting their outcomes, generating trajectories, evaluating their rewards, and selecting the best action**.

The key flow is:

Current State → Sample Actions → Environment Model → Trajectories → Evaluation → Best Action

It is especially useful in applications such as **robotics, autonomous vehicles, game AI, industrial automation, and navigation**, where considering every possible future would require excessive computation.

2. Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

(5 Marks)

Concept Map Showing How Sampled Trajectories Are Generated and Evaluated for Planning

1. Introduction

In **Model-Based Reinforcement Learning**, the agent uses an environment model to predict what may happen after taking an action. Instead of exploring every possible future, **sampling-based planning** generates a limited number of possible trajectories and evaluates them.

A **trajectory** is a sequence of states and actions followed by the agent:

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

where s_t represents the state at time t, a_t represents the action, and T is the planning horizon.

The planner generates several candidate trajectories, calculates their expected rewards, compares them, and selects the most promising trajectory.

2. Basic Concept

The overall process is:

Current State → Sample Actions → Predict States → Generate Trajectories → Calculate Rewards → Evaluate → Compare → Select Best Trajectory

For example, suppose a robot wants to reach a target. The planner can sample different paths:

- Trajectory 1 → Move forward → Turn left → Reach target
- Trajectory 2 → Move forward → Move forward → Reach target
- Trajectory 3 → Turn right → Move forward → Collision

The planner evaluates these trajectories and selects the safest and most rewarding one.

. Generation of Sampled Trajectories

Trajectory generation begins with the **current state** of the agent.

Let the current state be:

$$s_0$$

The planner samples an action:

$$a_0$$

The environment model predicts the next state:

$$s_1 = f(s_0, a_0)$$

Another action is then sampled:

$$a_1$$

and the model predicts:

$$s_2 = f(s_1, a_1)$$

This continues until a specified planning horizon is reached.

Therefore, one trajectory can be written as:

$$\tau_1 = (s_0, a_0, s_1, a_1, s_2, \dots, s_T)$$

Multiple trajectories are generated:

$$\tau_1, \tau_2, \tau_3, \dots, \tau_N$$

Each trajectory represents a different possible future.

4. Steps Involved in Trajectory Generation

Step 1: Obtain Current State

The agent observes the current environment and obtains s_0.

Step 2: Sample an Action

The planner chooses one possible action from the available action space.

Step 3: Predict the Next State

The environment model predicts the state resulting from the selected action.

Step 4: Repeat Sampling

Another action is sampled based on the predicted state.

Step 5: Continue Until Planning Horizon

Actions and states are generated repeatedly until the required number of steps is reached.

Step 6: Store the Trajectory

The complete sequence of states and actions is stored for evaluation.

5. Evaluation of Sampled Trajectories

After generating trajectories, the planner must determine which trajectory is best.

The main factor used for evaluation is the **expected return**.

The cumulative discounted return is:

$$G = \sum_{t=0}^{T-1} \gamma^t r_t$$

where:

- G= total return
- r_t= reward at time t
- γ= discount factor
- T= planning horizon

A higher return generally indicates a better trajectory.

The planner may also consider:

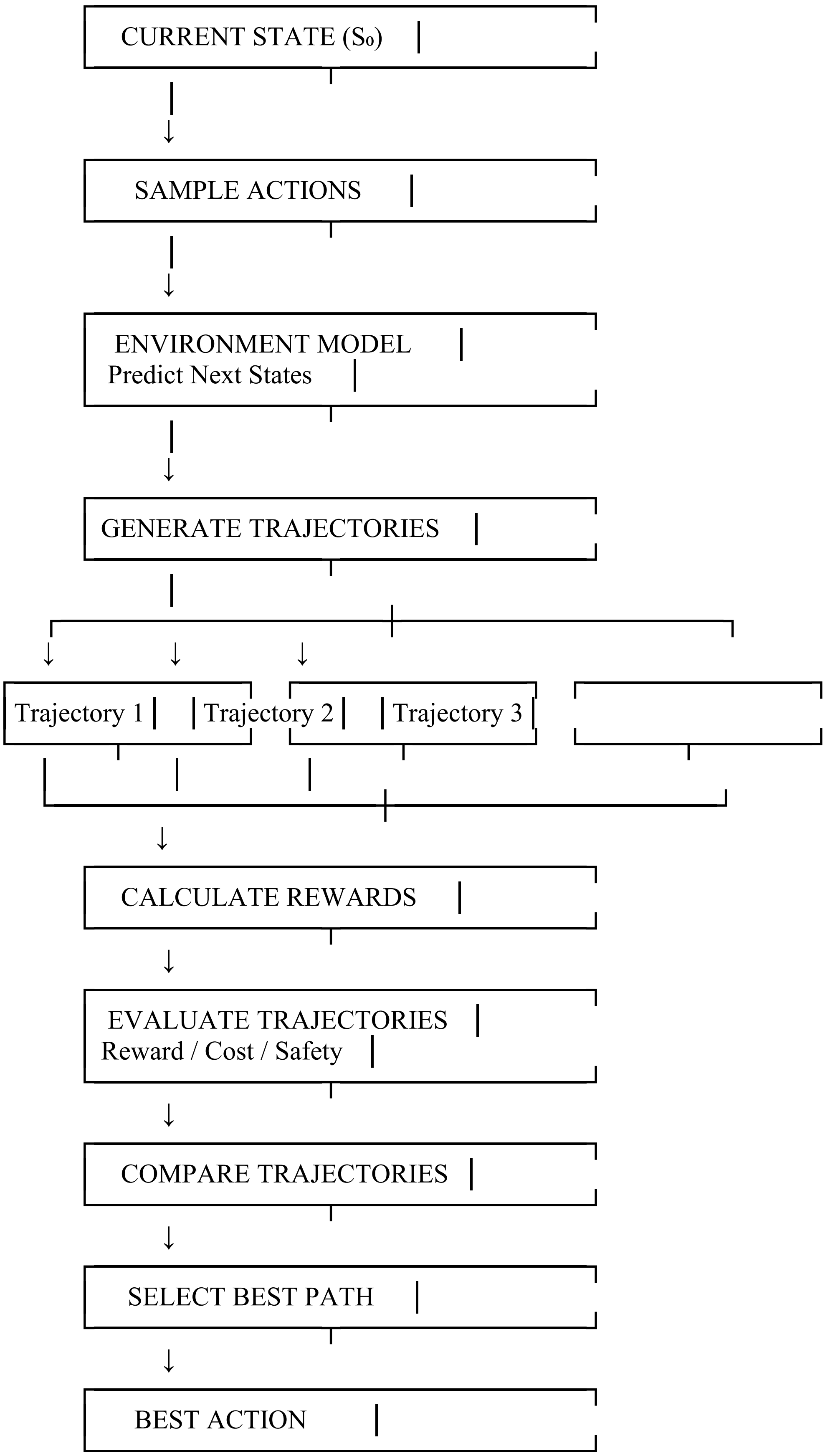
- Safety
- Distance to goal
- Time taken
- Energy consumption
- Collision risk
- Resource usage

Thus, trajectory evaluation can use a combined reward or cost function.

6. Concept Map

SAMPLING-BASED PLANNING

↓



7. Trajectory Evaluation Example

Consider an autonomous robot that needs to reach a destination.

The planner samples three trajectories.

Trajectory	Distance	Safety	Reward
T1	Long	High	70
T2	Medium	High	90
T3	Short	Low	45

The planner does not simply select the shortest trajectory.

Although **T3** has the shortest distance, it has a low safety score. **T2** provides a good balance between distance and safety and has the highest reward.

Therefore:

$$T^* = \arg\max_{\tau} G(\tau)$$

So, **Trajectory T2 is selected**.

. Role of the Environment Model

The environment model is an important component because it allows the planner to simulate possible futures.

It can be represented as:

$$P(s_{t+1}, \mid s_t a_t)$$

This represents the probability of reaching the next state given the current state and action.

For example, in a robot navigation system, the model can predict:

Current Position + Movement Action → Predicted Position

Without an environment model, the planner would have difficulty predicting the consequences of actions before executing them.

9. Role of Sampling

Sampling reduces the amount of computation required.

Suppose an environment has:

- 5 possible actions
- Planning horizon = 6

An exhaustive search could require:

$$5^6 = 15{,}625$$

possible action sequences.

Instead, sampling-based planning might evaluate only 100 selected trajectories.

This significantly reduces the search effort while still allowing the planner to find a good solution.

10. Advantages of Sampled Trajectory Evaluation

1. Computational Efficiency

Only a limited number of trajectories are evaluated.

2. Suitable for Large State Spaces

It can be applied when the environment has many possible states.

3. Supports Continuous Actions

Sampling can be used for continuous actions such as steering angles or robot movement.

4. Future Prediction

The planner can evaluate possible future outcomes before executing an action.

5. Flexible Evaluation

Trajectories can be evaluated using multiple objectives such as safety, speed, cost, and reward.

11. Limitations

Sampling-based trajectory evaluation also has some limitations.

Limited Coverage

The sampled trajectories may not cover the entire search space.

Possible Miss of Optimal Solution

The globally optimal trajectory may not be sampled.

Computational Cost

A large number of samples can still require significant computation.

Model Accuracy

If the environment model makes incorrect predictions, trajectory evaluation may also be inaccurate.

12. Real-Life Application

A good example is **autonomous vehicle navigation**.

The vehicle observes its current position and surrounding traffic. The planner samples possible driving trajectories such as:

- Continue straight
- Slow down
- Change lane
- Turn

The environment model predicts the future position of the vehicle for each option.

The trajectories are evaluated based on:

- Collision risk
- Distance
- Traffic
- Travel time
- Comfort
- Energy consumption

The safest high-reward trajectory is selected, and the vehicle executes its first action.

After the vehicle moves, it obtains a new state and repeats the planning process.

3. Conclusion

Sampled trajectory generation and evaluation are important parts of sampling-based planning in Model-Based Reinforcement Learning. The planner starts with the current state, samples possible actions, uses the environment model to predict future states, and creates multiple trajectories.

Each trajectory is then evaluated using rewards, costs, safety, distance, or other objectives. The trajectory with the highest expected return is selected, and its best action is executed.

Key Flow

Current State → Sample Actions → Predict States → Generate Trajectories → Evaluate → Compare → Selec

Thus, sampled trajectory planning provides a practical balance between **planning quality and computational efficiency**, making it useful in robotics, autonomous vehicles, games, and intelligent control systems.

3. **Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.**

(5 Marks)

Block Diagram: Interaction Between Environment Model, Planner, Sampled Actions, and Optimal Policy

Basic Idea

In Model-Based Reinforcement Learning, the **planner** uses the **environment model** to predict the results of sampled actions. It evaluates the possible outcomes and selects the action that gives the best expected reward. This forms the **optimal policy**.

Block Diagram

Environment

↓ Current State

Planner

↓

Sampled Actions

↓

Environment Model

↓ Predicted States

Evaluate Trajectories

↓

Select Best Action

↓

Optimal Policy

↓

Environment

Explanation

- **Environment:** Provides the current state to the agent.
- **Planner:** Generates possible actions for planning.
- **Sampled Actions:** Represents different possible actions or action sequences.
- **Environment Model:** Predicts future states for the sampled actions.
- **Trajectory Evaluation:** Calculates the reward or cost of each possible trajectory.
- **Best Action:** The action with the highest expected reward is selected.
- **Optimal Policy:** Determines the best action to take for a given state.

Example

For a **robot**, the planner may sample *move left, move right,* and *move forward*. The environment model predicts the result of each action. After comparing their rewards and safety, the robot selects the best action according to the optimal policy.

4. Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

(5 Marks)

Flowchart: Monte Carlo Sampling for Planning in Reinforcement Learning

Basic Idea

Monte Carlo sampling uses repeated simulations or sampled episodes to estimate the expected reward of different actions. The action with the highest estimated return is selected.

Flowchart

START

↓

Observe Current State

↓

Sample an Action

↓

Simulate Episode / Trajectory

↓

Calculate Total Return / Reward

↓

Repeat Sampling

↓

Average Returns

↓

Select Action with Highest Expected Reward

↓

END

Short Explanation

1. **Observe State:** The agent observes its current state.
2. **Sample Action:** A possible action is selected.
3. **Simulate:** The action is followed through a simulated trajectory.
4. **Calculate Return:** The total reward obtained is calculated.
5. **Repeat:** Multiple trajectories are sampled.
6. **Average Returns:** The returns from the samples are averaged.
7. **Select Best Action:** The action with the highest expected return is chosen.

The estimated value can be represented as:

$$V(s) \approx \frac{1}{N}\sum_{i=1}^{N} G_i$$

where N is the number of samples and G_i is the return from each sampled episode.

Example: A robot samples three paths with rewards 50, 80, and 65. It selects the path with reward **80** because it provides the highest expected return.

5. Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

(5 Marks)

Basic Idea

Both techniques are used to find a good action or trajectory in Reinforcement Learning.

```
                PLANNING TECHNIQUES
                      |
      ┌───────────────┴───────────────┐
      ↓               ↓
EXHAUSTIVE SEARCH        SAMPLING-BASED
  PLANNING                 PLANNING
      |               |
      ↓               ↓
Explore ALL          Explore SOME
possibilities       possibilities
      |               |
      ↓               ↓
 Large search        Reduced search
   space             space
      |               |
      ↓               ↓
High computation        Less computation
      |               |
      ↓               ↓
 More complete        Approximate
   solution          solution
      |               |
      ↓               ↓
Best for small       Best for large/
search spaces         continuous spaces
      |               |
      └───────────────┬───────────────┘
              ↓
       Evaluate Trajectories
              ↓
        Select Best Action
```

Comparison

Feature	Exhaustive Search	Sampling-Based Planning
Search	All possible paths	Selected paths
Computation	High	Lower
Speed	Slow for large spaces	Faster
Solution	More complete	Approximate
Large state space	Difficult	Suitable
Continuous actions	Difficult	Suitable
Main advantage	Thorough search	Efficient search
Main limitation	Computationally expensive	May miss optimal path

Example

Suppose a robot has **5 possible actions** for 6 steps.

Exhaustive search may examine:

$$5^6 = 15{,}625$$

possible action sequences.

Sampling-based planning may examine only **100 sampled trajectories** and select the one with the highest reward.

Conclusion

Exhaustive search provides a more complete search but requires high computation. **Sampling-based planning** reduces computation by examining only a subset of possibilities, making it more suitable for large and complex Reinforcement Learning problems.

Code of Conduct:

I Malli Chandana(192472250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : Malli Chandana

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness